Programiz C Online Compiler

main.c

Run

```c
#include <stdio.h>

#define N 6

int main()
{
    int graph[N][N] = {
        {0, 1, 1, 0, 0, 0},
        {1, 0, 0, 1, 1, 0},
        {1, 0, 0, 0, 0, 1},
        {0, 1, 0, 0, 0, 0},
        {0, 1, 0, 0, 0, 1},
        {0, 0, 1, 0, 1, 0}
    };

    char *location[N] = {
        "Hospital",
        "School",
        "Market",
        "Park",
        "Bus Stand",
        "Railway Station"
    };

    int visited[N] = {0};
    int queue[N];
    int front = 0, rear = 0;
    int start, current, i;
```

Type here to search

Programiz C Online Compiler

main.c

Run

```c
    int visited[N] = {0};
    int queue[N];
    int front = 0, rear = 0;
    int start, current, i;

    printf("CITY ROUTE USING BFS\n");
    printf("--------------------\n");

    printf("0 - Hospital\n");
    printf("1 - School\n");
    printf("2 - Market\n");
    printf("3 - Park\n");
    printf("4 - Bus Stand\n");
    printf("5 - Railway Station\n");

    printf("\nEnter starting location (0-5): ");

    if (scanf("%d", &start) != 1 ||
        start < 0 || start >= N)
    {
        printf("Invalid input!\n");
        return 1;
    }

    visited[start] = 1;
    queue[rear++] = start;
```

Type here to search

Programiz C Online Compiler

main.c

```
        return 1;
    }

    visited[start] = 1;
    queue[rear++] = start;

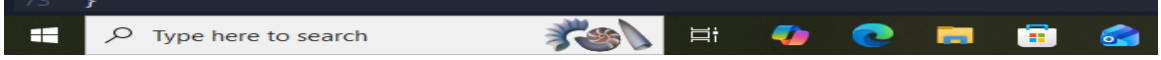
    printf("\nBFS Traversal Order:\n");

    while (front < rear)
    {
        current = queue[front++];
        printf("%s -> ", location[current]);

        for (i = 0; i < N; i++)
        {
            if (graph[current][i] == 1 &&
                visited[i] == 0)
            {
                visited[i] = 1;
                queue[rear++] = i;
            }
        }
    }

    printf("END\n");

    return 0;
}
```



Programiz PRO >

Output

```
CITY ROUTE USING BFS
--------------------
0 - Hospital
1 - School
2 - Market
3 - Park
4 - Bus Stand
5 - Railway Station

Enter starting location (0-5): 2

BFS Traversal Order:
Market -> Hospital -> Railway Station -> School -> Bus Stand -> Park -> END
```